

Trouve ton Artisan !

Plateforme de mise en relation avec des artisans
Région Auvergne-Rhône-Alpes

Étudiant : Florent Vidal

Formation : CEF — Développement Web

Date : 18/03/2026

GitHub : github.com/ldhZY/trouve-ton-artisan

1. Présentation du projet

Trouve ton Artisan est une application web permettant aux particuliers de la région Auvergne-Rhône-Alpes de trouver et contacter des artisans qualifiés. L'utilisateur peut naviguer par catégorie d'artisanat, rechercher un artisan par nom, consulter ses informations détaillées et lui envoyer un message directement via un formulaire de contact.

Fonctionnalités principales :

- Navigation par catégories (Alimentation, Bâtiment, Fabrication, Services)
- Recherche d'artisans par nom (autocomplétion en temps réel)
- Fiche détaillée par artisan : note, spécialité, ville, code postal, site web, description
- Formulaire de contact avec envoi d'email via Nodemailer
- Section « Artisans du mois » sur la page d'accueil
- Pages légales : Mentions légales, Données personnelles, Accessibilité, Cookies
- Design responsive — mobile, tablette, desktop

2. Stack technique

Backend

Environnement	Node.js 22
Framework	Express 5
ORM	Sequelize 6
Base de données	MySQL 9
Email	Nodemailer (Gmail SMTP)
Hébergement	Railway (cloud)

Frontend

Framework	React 19
Build tool	Vite 8
Routing	React Router v7
Style	Sass + Bootstrap 5
Hébergement	Vercel

Outils & DevOps

Versionning	Git / GitHub
Branche prod.	main
CI/CD backend	Railway — déploiement auto sur push main
CI/CD frontend	Vercel — déploiement auto sur push main
BDD Railway	MySQL 9.6 — host shinkansen.proxy.rlwy.net:19639

3. Architecture du projet

Structure des dossiers

```
trouve-ton-artisan/ ← racine du dépôt
  ■■■■ config/
  ■ ■■■■ database.js ← connexion Sequelize
  ■■■■ middleware/
  ■ ■■■■ auth.js ← vérification clé API (x-api-key)
  ■■■■ models/
  ■ ■■■■ artisan.js
  ■ ■■■■ categorie.js
  ■ ■■■■ specialite.js
  ■ ■■■■ index.js ← associations ORM
  ■■■■ routes/
  ■ ■■■■ RoutesArtisan.js ← 6 routes REST
  ■■■■ server.js ← Express + CORS + middleware
  ■■■■ .env ← variables d'environnement (non versionné)
  ■■■■ trouve-ton-artisan/ ← sous-dossier frontend
  ■■■■ src/
  ■ ■■■■ components/ ← Header, Footer
  ■ ■■■■ pages/ ← Home, ArtisanList, ArtisanDetail, Legal, NotFound
  ■ ■■■■ services/
  ■ ■ ■■■■ api.js ← appels fetch vers le backend
  ■ ■■■■ styles/ ← fichiers .scss
  ■■■■ .env.production ← URL Railway pour production
  ■■■■ vercel.json ← rewrites SPA
```

4. Base de données

La base de données MySQL comporte 3 tables liées par des clés étrangères. Sequelize gère les associations via des relations **belongsToMany** / **hasMany**.

Table categorie

id	INT UNSIGNED — clé primaire auto-incrémentée
nom	VARCHAR(100) — nom de la catégorie

Table specialite

id	INT UNSIGNED — clé primaire
nom	VARCHAR(100) — nom de la spécialité
id_categorie	INT UNSIGNED — clé étrangère → categorie.id

Table artisan

id	INT UNSIGNED — clé primaire
nom	VARCHAR(150)
note	DECIMAL(2,1) — note sur 5
ville	VARCHAR(100)
code_postal	VARCHAR(10) — nullable
a_propos	TEXT — description libre, nullable
email	VARCHAR(255) — pour l'envoi de messages
site_web	VARCHAR(255) — nullable
top_artisan	TINYINT — 1 = artisan du mois
id_specialite	INT UNSIGNED — clé étrangère → specialite.id

Relations

Categorie → Specialite	hasMany — une catégorie possède plusieurs spécialités
Specialite → Artisan	hasMany — une spécialité regroupe plusieurs artisans
Artisan → Specialite	belongsToMany — un artisan appartient à une spécialité

5. API Backend — Routes REST

Toutes les routes sont préfixées par **/api/artisans** et protégées par un middleware d'authentification via l'en-tête **x-api-key**.

Méthode	Route	Description
GET	/categories	Liste des catégories triées alphabétiquement
GET	/top/artisans	3 artisans du mois (top_artisan = 1)
GET	/categorie/:id	Artisans filtrés par id_categorie (via jointure Specialite)
GET	/recherche/:nom	Recherche par nom (LIKE %nom%)
GET	/:id	Détail d'un artisan + sa spécialité
POST	/:id/contact	Envoi d'email à l'artisan (nom, email, objet, message)

Sécurité

Authentification	En-tête x-api-key vérifié sur chaque requête (middleware auth.js)
CORS	Origines autorisées : localhost:5173, trouve-ton-artisan-liart.vercel.app, my-project-pi-neon.vercel.app
Validation	Vérification des champs obligatoires sur POST /contact

6. Frontend React

Pages

Page	Route	Description
Home	/	Accueil : étapes, artisans du mois
ArtisanList	/categorie/:slug	Liste des artisans par catégorie
ArtisanList	/recherche?q=...	Résultats de recherche
ArtisanDetail	/artisan/:id	Fiche complète + formulaire contact
Legal	/mentions-legales etc.	Pages légales (4 routes)
NotFound	*	Page 404

Composants

- **Header** — Logo, navigation par catégories (chargées depuis l'API), barre de recherche avec autocomplétion (debounce 300ms), menu burger mobile
- **Footer** — Adresse de la région, liens légaux
- **StarRating** — Composant d'affichage des étoiles (utilisé dans ArtisanList et ArtisanDetail)

Service API (src/services/api.js)

Toutes les requêtes transitent par une fonction **apiFetch()** qui injecte automatiquement l'en-tête **x-api-key** et l'URL de base depuis les variables d'environnement Vite (**VITE_API_URL** / **VITE_API_KEY**).

7. Déploiement

Backend — Railway

URL de production	<code>https://trouve-ton-artisan-production-10b5.up.railway.app</code>
Base de données	MySQL 9.6 — base <code>trouve_ton_artisan</code> sur Railway
Start command	<code>node server.js</code>
Branche surveillée	<code>main</code> — déploiement automatique sur push
Variables Railway	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>PORT</code> , <code>API_KEY</code> , <code>MAIL_*</code>

Frontend — Vercel

URL de production	<code>https://my-project-pi-neon.vercel.app</code>
Root Directory	<code>trouve-ton-artisan</code>
Build command	<code>npm run build</code> (vite build)
Branche surveillée	<code>main</code> — déploiement automatique sur push
Variables Vercel	<code>VITE_API_URL</code> , <code>VITE_API_KEY</code> (via <code>.env.production</code>)
Routing SPA	<code>vercel.json</code> — rewrites <code>/*</code> → <code>/index.html</code>

8. URLs et accès

Site en production	https://my-project-pi-neon.vercel.app
API backend	https://trouve-ton-artisan-production-10b5.up.railway.app/api
Dépôt GitHub	https://github.com/ldhZY/trouve-ton-artisan
Clé API (x-api-key)	tta_9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c

Exemples d'appels API

GET /api/artisans/categories

GET /api/artisans/top/artisans

GET /api/artisans/categorie/1

GET /api/artisans/recherche/boulangier

GET /api/artisans/3

POST /api/artisans/3/contact { nom, email, objet, message }

9. Bilan

Points forts

- Architecture REST claire et sécurisée (clé API, CORS restrictif)
- Application full-stack complète déployée en production
- Interface responsive et accessible (skip-link, aria-labels, fil d'Ariane)
- Recherche en temps réel avec debounce
- Formulaire de contact fonctionnel avec envoi d'email réel
- Code organisé en couches (models, routes, middleware, services)

Améliorations possibles

- Ajouter les codes postaux dans les données artisans
- Pagination sur la liste des artisans
- Système d'authentification pour les artisans
- Upload de photo de profil
- Tests automatisés (Jest, Vitest)